

Teórica 7
Padrões de Desenho de Software

Porque é que os programadores confundem o Natal com o Halloween?



Sumário

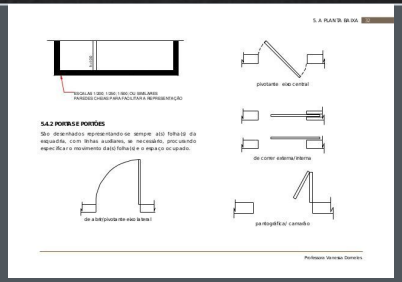
- Revisitando a definição de Desenho
  - Desenho Funcional
  - Desenho orientado a objectos
- Definição de padrões
- Padrões de Desenho GoF
  - Singleton
  - Facade
  - Object Factory
  - Factory Method

Revisitando o desenho

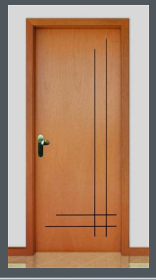
Desenho

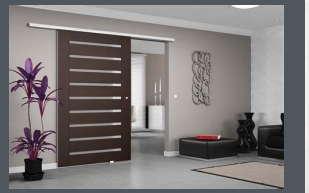
Requisitos: Mecanismo que nos permite passar por paredes.

Desenho:



Implementação:







Desenho

O processo de passar do domínio do problema para o domínio da solução.

Este processo produz uma descrição de uma solução do problema que satisfaz os requisitos.

A fase de Desenho do software envolve dois tipos de artefactos de Desenho

- Arquitectura de software – descreve ao cliente o que faz o sistema, e como propriedades como o desempenho, segurança e disponibilidade são influenciadas
  - Descreve as funções do sistema
  - Relacionado com os requisitos

  Voltaremos a este tópico mais tarde na cadeira
- Desenho do programa – descreve à equipa de programadores como o sistema deve funcionar
  - Descrição dos algoritmos principais
  - Estruturas de dados e fluxo de dados

Técnicas de Desenho

Decomposição

Decomposição funcional – modela o sistema como um conjunto de funcionalidades, decompondo-as noutras mais simples. Presume-se que os dados estejam centralizados e não sejam acessíveis por todas as funcionalidades.

Desenho orientado a objectos – modela o sistema com um conjunto de objectos que interagem entre si. Cada objecto encapsula parte das funcionalidades e regula o acesso a parte da informação.

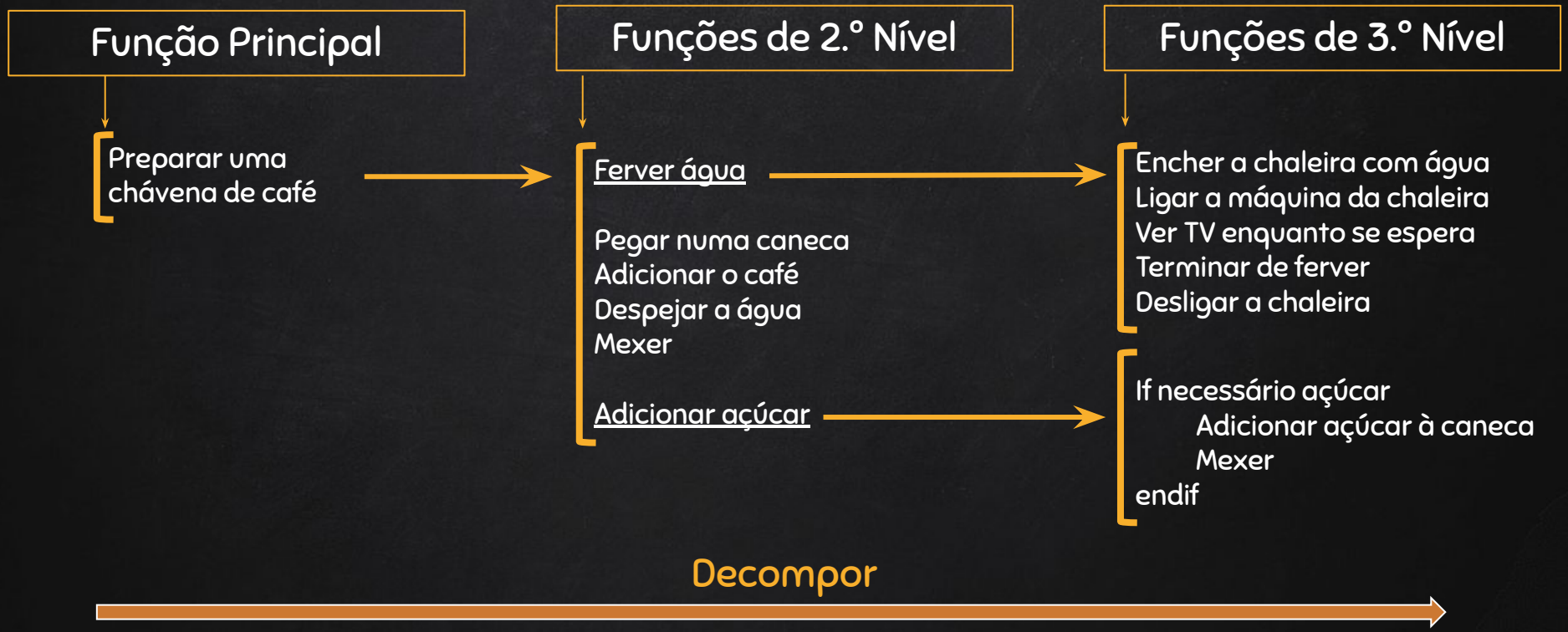
Desenho Funcional

Concentra-se nas funções ou acções e na decomposição funcional que o sistema deve suportar

Passos:

1. Descrever a função principal
2. Decompor a função principal em subfunções
3. Decompor cada subfunção em subfunções mais detalhadas
4. Repetir o processo até que seja fácil de traduzir a subfunção em código

Exemplo


Função Principal
Funções de 2.º Nível
Funções de 3.º Nível
Preparar uma chávena de café
Ferver água
Pegar numa caneca
Adicionar o café
Despejar a água
Mexer
Adicionar açúcar
Encher a chaleira com água
Ligar a máquina da chaleira
Ver TV enquanto se espera
Terminar de ferver
Desligar a chaleira
If necessário açúcar
Adicionar açúcar à caneca
Mexer
endif
Decompor

Características da decomposição funcional

Abordagem descendente

Abordagem de refinamento passo-a-passo, diminuindo gradualmente a abstração

Decompõe o sistema em funções

As funções partilham um estado do sistema global e centralizado

As funções podem ter um estado local, apenas durante a execução

Desenho funcional

Vantagens

- Pode ser usado como estratégia para qualquer aplicação de software
- Flexível para o desenho
- Forma natural de orientar os pensamentos

Desvantagens

- Considerações sobre dados são ignoradas ou adiadas
- As funções de dados podem comprometer o desenho
- Os detalhes sobre os algoritmos estão no nível da função, mas o estado não está oculto
    - Uma função que altera o estado global pode ter um impacto colateral noutras funções

Desenho Orientado a Objectos

Princípios

- Encapsulamento – restringe o acesso a representações internas
- Abstração – remove detalhes desnecessários, destacando o essencial
- Modularidade – construída com base em elementos mais refinados.

Divide um sistema em módulos, onde cada módulo funciona de forma independente.

Primitivas

- Objetos, classes, herança, agregação

Etapas orientadas a objectos (explicadas a seguir)

É necessário ter em conta algumas etapas para desenvolver um projecto de sistema, desde o conceito até ao projecto OO, nomeadamente:

1. Compreender e definir o contexto e as interações externas com o sistema.
2. Projectar a arquitetura do sistema.
3. Identificar os principais objetos no sistema.
4. Desenvolver modelos de desenho.
5. Especificar interfaces.

Vamos ver o que cada etapa significa com um exemplo...

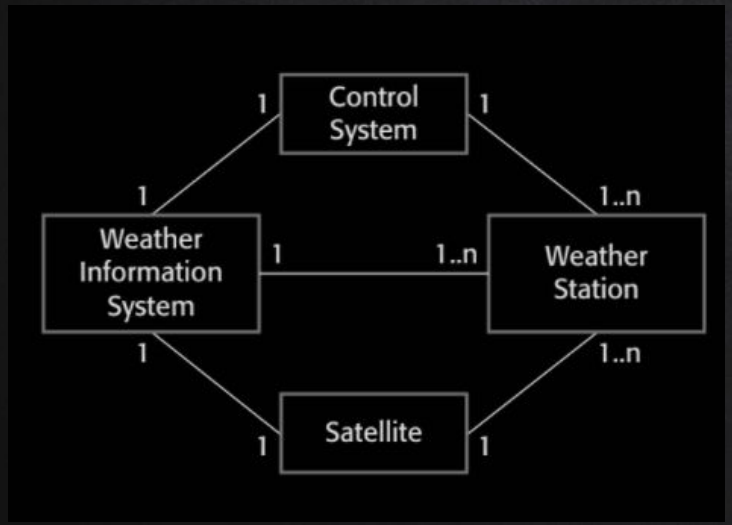
1. Contexto e interações do sistema

Os modelos de contexto e os modelos de interação do sistema apresentam visões complementares das relações entre um sistema e o seu ambiente:

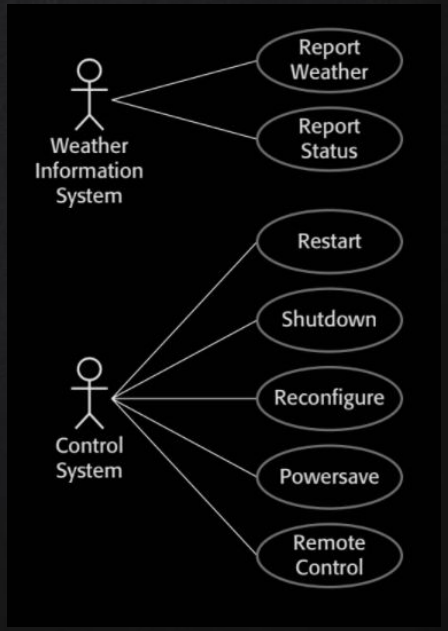
- Um modelo de contexto é um modelo estrutural que posiciona os outros sistemas no ambiente do sistema que está a ser desenvolvido.
- Um modelo de interação é um modelo dinâmico que mostra como o sistema interage com o seu ambiente à medida que é utilizado.

Exemplo de sistema de estação meteorológica

Modelo de contexto do sistema



Modelo de interação do sistema (modelos de caso de uso)



Descrição detalhada do caso de uso

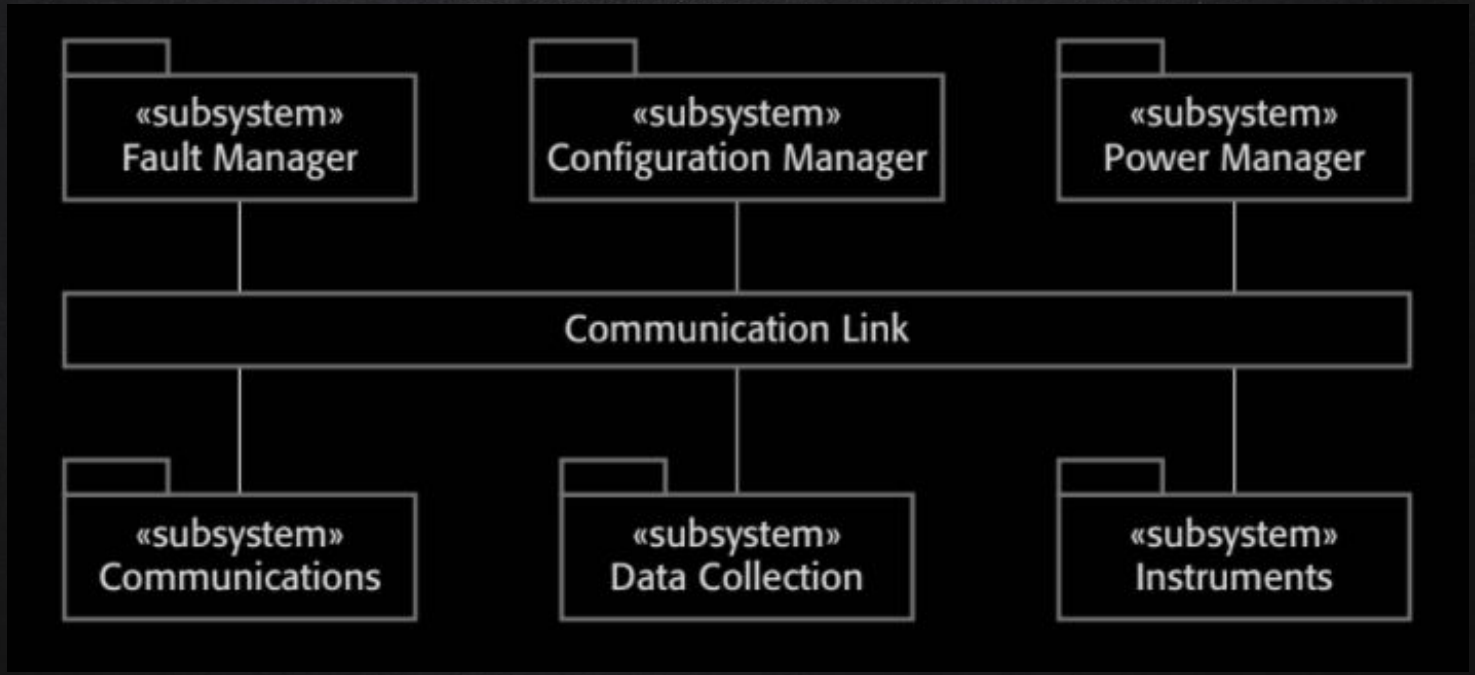
System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Dat	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum, and average wind speeds; the total rainfall; and the wind direction as sampled at five-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data are sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in the future.

2. Projeto arquitectural

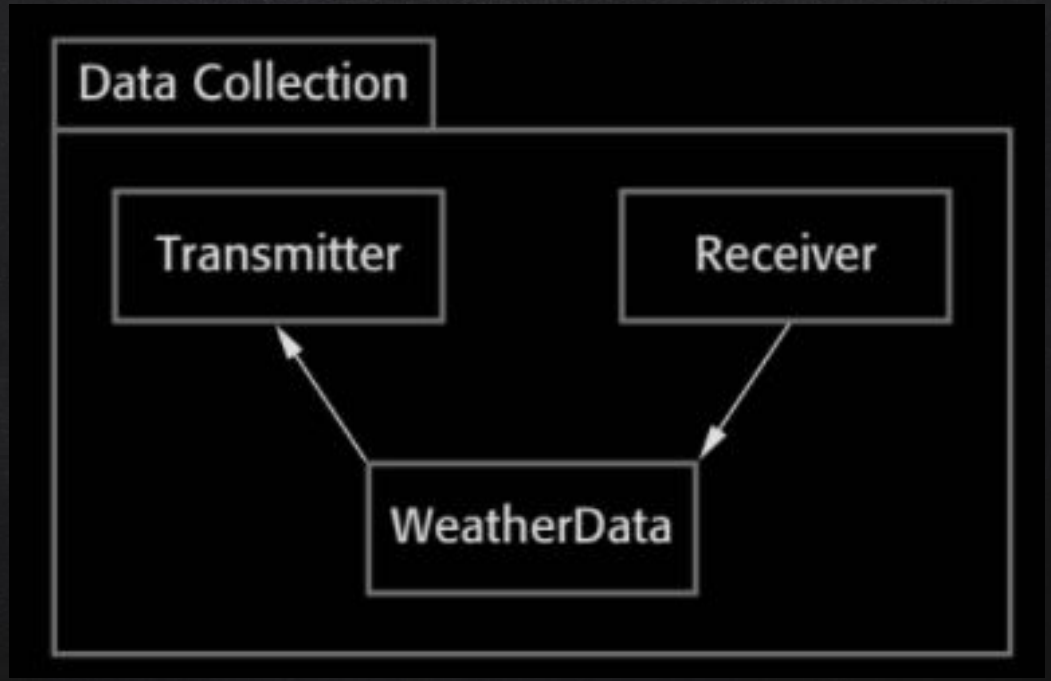
Depois de definidas as interações entre o sistema de software e o ambiente, utiliza-se essa informação como base para projectar a arquitectura do sistema.

Tendo um conhecimento geral dos princípios do projeto arquitectural e um conhecimento mais detalhado sobre o domínio, identificam-se os principais componentes que compõem o sistema e as suas interações e, em seguida, organizam-se os componentes utilizando um padrão arquitectónico.

Modelos de subsistemas



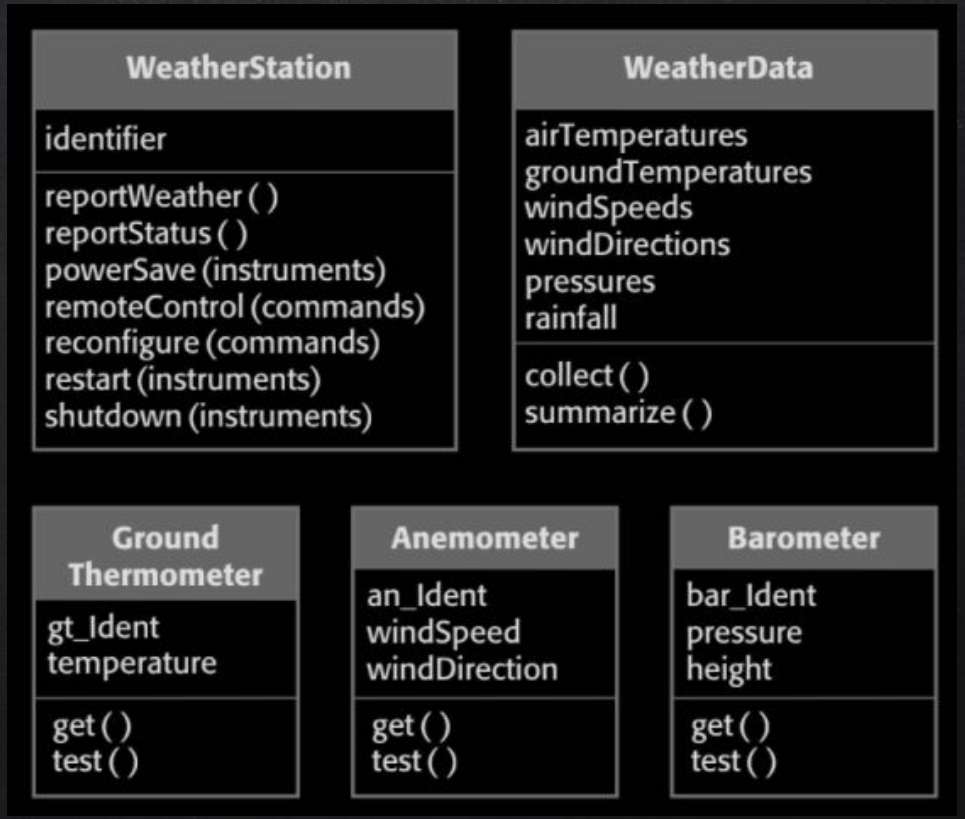
Detalhando o subsistema de recolha de dados



3. Identificar as classes de objectos

Nesta fase do processo de desenho, já existem algumas ideias sobre os objetos essenciais do sistema a projetar.

À medida que a compreensão do desenho se desenvolve, refinam-se essas ideias sobre os objetos do sistema.


WeatherStation
identifier
reportWeather ( )
reportStatus ( )
powerSave (instruments)
remoteControl (commands)
reconfigure (commands)
restart (instruments)
shutdown (instruments)
WeatherData
airTemperatures
groundTemperatures
windSpeeds
windDirections
pressures
rainfall
collect ( )
summarize ( )
Ground Thermometer
gt_Ident
temperature
get ( )
test ( )
Anemometer
an_Ident
windSpeed
windDirection
get ( )
test ( )
Barometer
bar_Ident
pressure
height
get ( )
test ( )

4. Modelos de Desenho

Os modelos de desenho ou de sistema mostram objectos, ou classes de objectos, do sistema.

Também descrevem associações e relações entre as entidades.

Estes modelos são a ponte entre os requisitos e a implementação do sistema.

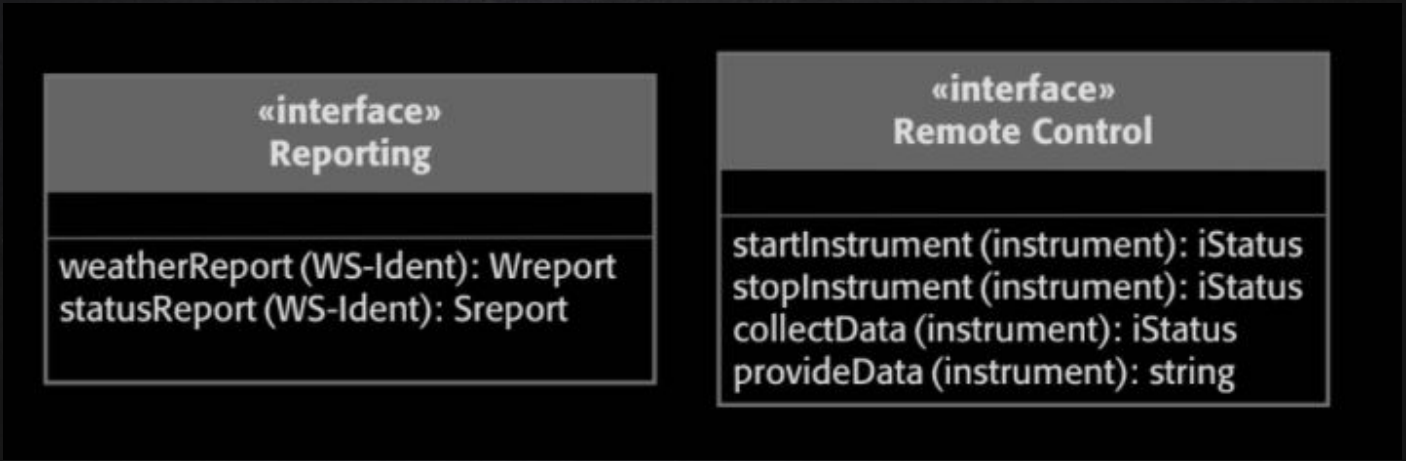
Normalmente, os tipos de modelos UML usados neste nível são:

- Diagramas de classes (Estrutural)
- Diagramas de interação (Comportamento)
- Diagramas de máquina de estados (Comportamento)

5. Especificação da Interface

Especificar os detalhes da interface para um objeto ou um grupo de objetos (como componente do sistema).

Significa definir as assinaturas e a semântica dos serviços fornecidos pelo objecto ou por um grupo de objectos.



Desenho com UML

1. Modelos Estruturais – descrevem a estrutura estática do sistema utilizando classes de objectos e suas relações. As relações importantes que podem ser documentadas nesta fase são relações de generalização (herança), relações de uso/utilização e relações de composição.
2. Modelos Dinâmicos – descrevem a estrutura dinâmica do sistema e mostram as interacções entre os objectos do sistema. As interações que podem ser documentadas incluem a sequência de pedidos ao serviço feitas por outros objectos e as mudanças de estado que são despoletadas pelos pedidos.

Alguns princípios de Desenho

- Independência (para facilitar a compreensão, adaptação e evolução)
  - Coesão
    - Uma classe ou entidade é coesa se todos os seus elementos estiverem envolvidos na realização dos seus objetivos

    Se não houver coesão, quando modificamos uma determinada função, é necessário procurar no resto do código as partes que fazem parte dessa função
  - Acoplamento
    - As classes ou entidades são acopladas quando os métodos de uma classe utilizam métodos ou variáveis de instância definidos numa classe diferente.

Se as entidades forem altamente dependentes, é mais provável que a alteração de uma classe afete outras entidades no programa.

Elevada coesão – Bom Desenho

Uma classe GestorDeEncomendas é responsável por criar encomendas, calcular os respetivos totais e verificar o seu estado.

Estas operações estão relacionadas com o mesmo conceito do domínio: a gestão de encomendas.

Baixa coesão – Mau Desenho

Uma classe Sistema gere encomendas, envia emails, calcula salários e gera gráficos.

As responsabilidades não estão relacionadas entre si, tornando a classe mais difícil de compreender, testar e modificar.

Elevado Acoplamento – Mau Desenho

```
class Encomenda {
    void confirmar() {
        // Confirmar a encomenda
        ServicoEmail email = new ServicoEmail();
        email.enviar("Encomenda confirmada");
    }
}
```

Encomenda depende diretamente de `ServicoEmail`. Se quisermos substituir o envio de email por uma notificação móvel, teremos de alterar a classe `Encomenda`. Dependência desnecessária.

Menor Acoplamento – Bom Desenho

Encomenda depende da abstração Notificador, em vez de depender diretamente de ServicoEmail.

Podemos fornecer-lhe diferentes implementações, como ServicoEmail ou NotificadorMovel, sem alterar a lógica de confirmação da encomenda.

```
class ServicoEmail implements Notificador {
    public void enviar(String mensagem) { ... }
}

class Encomenda {
    private final Notificador notificador;

    Encomenda(Notificador notificador) {
        this.notificador = notificador;
    }

    void confirmar() {
        // Confirmar a encomenda
        notificador.enviar("Encomenda confirmada");
    }
}
```

Este desenho reduz o acoplamento à implementação concreta e facilita os testes, a manutenção e a evolução do sistema.

Tipos de acoplamento

- Acoplamento de conteúdo – quando um elemento conhece a estrutura interna do outro
  - Qualquer alteração pode afetar os restantes elementos
- Acoplamento de partilha – quando vários elementos partilham um conjunto de dados (por exemplo, variáveis globais)
- Acoplamento de controlo – quando um elemento passa parâmetros que controlam a lógica de outro elemento (o elemento controlado não funciona sem o controlador)
- Ligação estrutural – quando uma estrutura de dados é utilizada para passar informação entre componentes (a organização e o formato dos dados estabelecem a dependência entre os elementos)
- Acoplamento de dados – os dados são trocados entre elementos (apenas dados simples)

Alguns princípios de Desenho

O desenho deve ter como objectivo:

- Coesão
  - Alta coesão
- Acoplamento
  - Baixo acoplamento

Mais princípios

- Manter as classes separadas e evitar que tenham muito conhecimento umas sobre as outras
- Encapsular tudo o que varia
- Preferir composição a herança
- Programar para interfaces, não para implementações

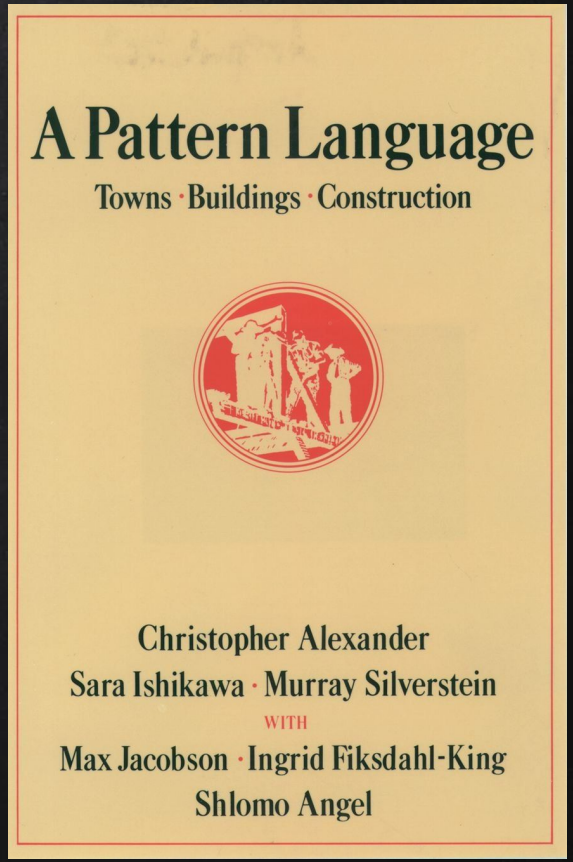
Padrões



Padrões

Inspirado no livro de arquitetura (projectos de edifícios) de Christopher Alexander, publicado em 1977

Uma solução geral e reutilizável para um problema comum



Padrões de Desenho

Evitar refazer o trabalho com problemas recorrentes de desenho

Encontrar o mesmo problema de desenho várias vezes

Existem várias formas alternativas de lidar com problemas de desenho

Algumas soluções são preferíveis a outras por serem mais flexíveis ou reutilizáveis



O que são padrões de desenho de software?

São vários princípios de desenho que tornam o código mais:

- Reutilizável
- Flexível
- Fácil de manter

Para além disto, inclui a identificação de desenhos de software problemáticos (code smells)

O que é um padrão de desenho?

- Solução prática comprovada para um problema de desenho recorrente
  - não apenas teórica – são soluções reais utilizadas em software industrial.
- Soluções previamente delineadas que programadores experientes costumam usar para resolver um problema de software
  - não é necessário criar uma solução a partir dos princípios básicos da programação orientada a objetos cada vez que ocorre um problema
- Pode ser visto como um modelo de como resolver um problema que pode ser usado em muitas situações diferentes
- Usado para ajudar a corrigir código confuso e sem estrutura, também conhecido como "código esparguete".

Padrões de Desenho: o que NÃO são?



- NÃO são algoritmos que resolvem problemas computacionais
- NÃO são estruturas de dados utilizadas em classes e reutilizadas como estão (por exemplo, listas ligadas, tabelas de dispersão)
- NÃO é um desenho terminado que pode ser transformado diretamente em código
- NÃO são projectos complexos específicos de um domínio (para uma aplicação ou subsistema inteiro)

Nem sempre é óbvio

- Nem sempre é óbvio qual o padrão de desenho usar, pois existem muitos padrões de desenho

- Decidir qual usar é semelhante a um jogo de xadrez
  - existem muitos movimentos e estratégias possíveis para vencer
  - através da experiência e da análise da situação no tabuleiro, tornar-se-ão melhores a avaliar qual estratégia utilizar para ganhar um jogo ou resolver um problema.



Criando um vocabulário de desenho



Os padrões de desenho oferecem uma forma simples de discutir soluções de desenho, para que não seja necessário explicá-las repetidamente.

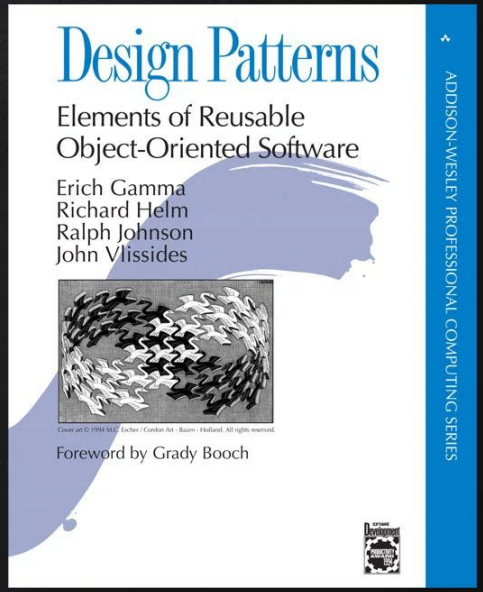
Por exemplo, os padrões de desenho podem nomear os padrões, facilitando a sua discussão. Isso poupa tempo e garante que todos se referem ao mesmo padrão, deixando menos margem para mal-entendidos.

Vamos usar UML, principalmente diagramas de classes, para nos ajudar a apresentar os padrões de desenho.

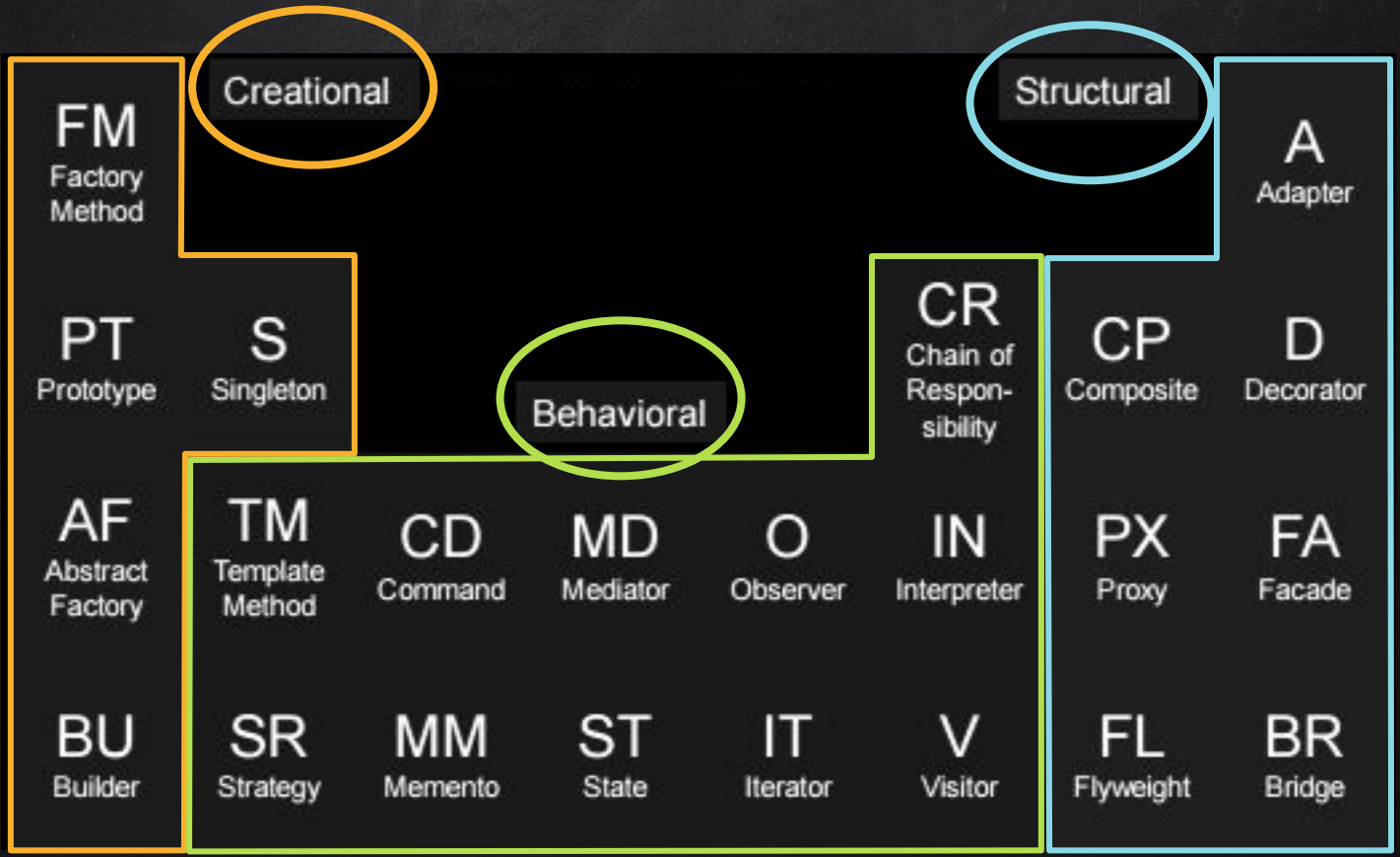
GoF – Gang of Four

Design Patterns: Elements of Reusable Object-Oriented Software

Erich Gamma, Richard Helm, Ralph Johnson, Craig Larman , John Vlissides, Addison Wesley Professional, 2004



Tipos de padrões



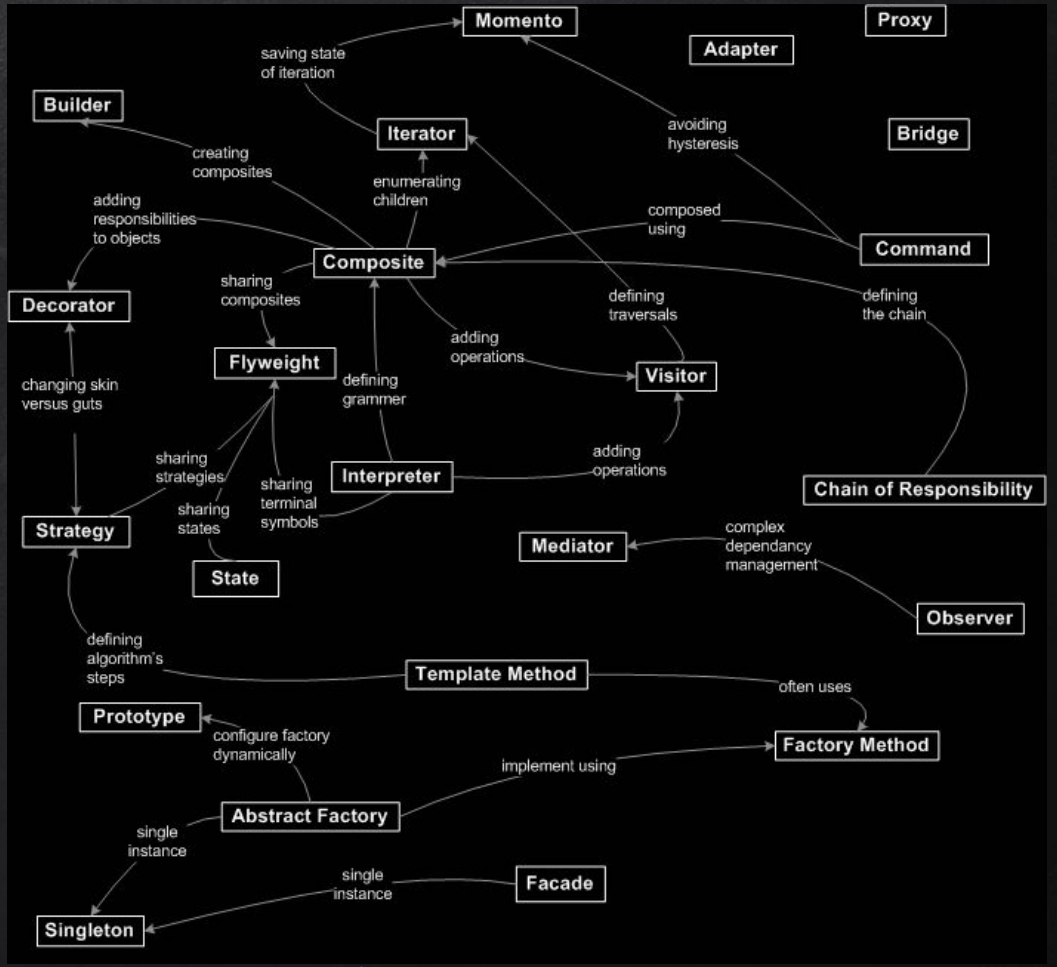
Tipos de padrões

Creational – lidam com a criação ou clonagem de novos objectos

Structural – descrevem como os objectos se ligam entre si

Behavioral – concentram-se na forma como os objectos distribuem o trabalho e descrevem como cada objecto executa uma única função coesa. Os padrões comportamentais também se concentram em como objetos independentes trabalham em prol de um objetivo comum.

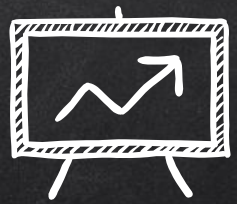
Relação entre padrões



Princípios dos padrões

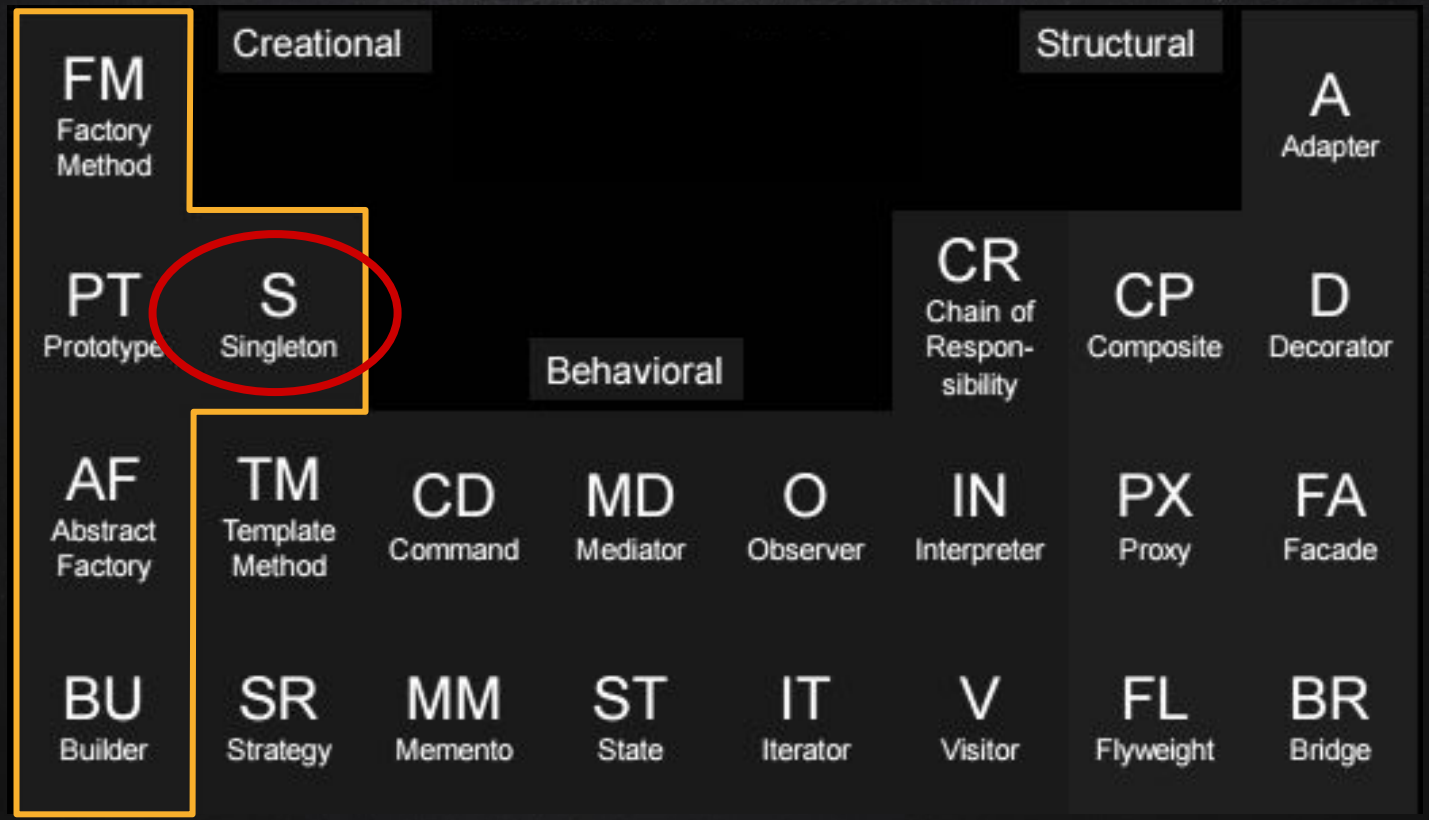
A razão para utilizar padrões de desenho é:

- Manter as classes separadas
- Evitar que tenham conhecimento excessivo umas sobre as outras



Desenho e Padrões de Software

Tipos de padrões



Padrão Singleton (Creational)

Descreve uma forma de criar um objeto. Uma classe tem apenas uma instância (ou um número limitado) e oferece um ponto global de acesso.



Problema: Como garantir que uma classe tem apenas uma instância e que essa instância está disponível?

- Uma variável global normal torna um objeto acessível, mas:
  - Polui o *namespace*
  - Ainda permite instanciar vários objetos dessa classe

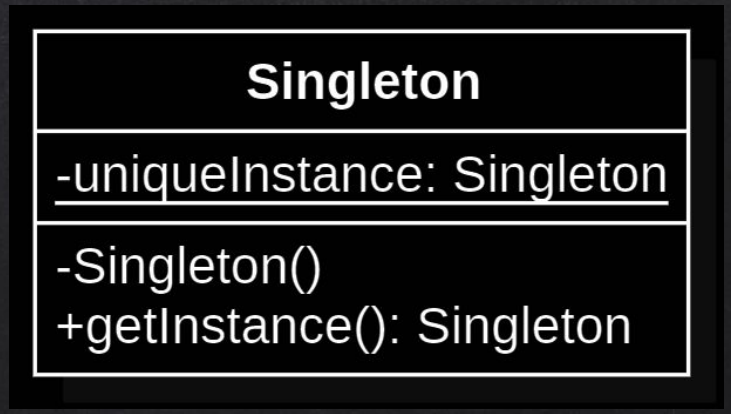
- Uma variável global estática torna um objeto acessível e não permite a instanciação múltipla, mas:
  - Não é possível controlar quem acede a essa variável
  - A variável é inicializada automaticamente ao carregar a classe

Melhor solução

Tornar a própria classe responsável por acompanhar a sua única instância

- A classe garante que nenhuma outra instância seja criada, interceptando solicitações para criar novos objetos
- A classe fornece a única forma de aceder à instância

Estrutura Singleton



Colaboração: os clientes acedem a uma instância Singleton exclusivamente através da operação getInstance() do Singleton

Exemplo Java Singleton

```
public class Singleton { // lazy construction

      // the class variable is null if no instance is instantiated
      private static Singleton uniqueInstance = null;

      private Singleton() {
            …
      }
      // lazy construction of the instance
      public static Singleton getInstance() {
            if (uniqueInstance == null) {
                  uniqueInstance = new Singleton();
            }
            return uniqueInstance;
      }
      …
}
```

Oferece acesso global a uma classe que está restrita a uma instância. Em geral, consegue-se através de um construtor privado, com um método público que instancia a classe, caso ela ainda não tenha sido instanciada.

Vantagens e desvantagens do Singleton

Vantagens:

Construção Lazy – o objecto não é criado até que seja realmente necessário. Isto é útil, especialmente se o objeto for grande. Como o objeto não é criado até que o método "`getInstance`" seja chamado, o programa é mais eficiente.

Desvantagens:

Várias *threads* em execução – podem ocorrer problemas causados pelas *threads* que tentam aceder ao único objeto partilhado.

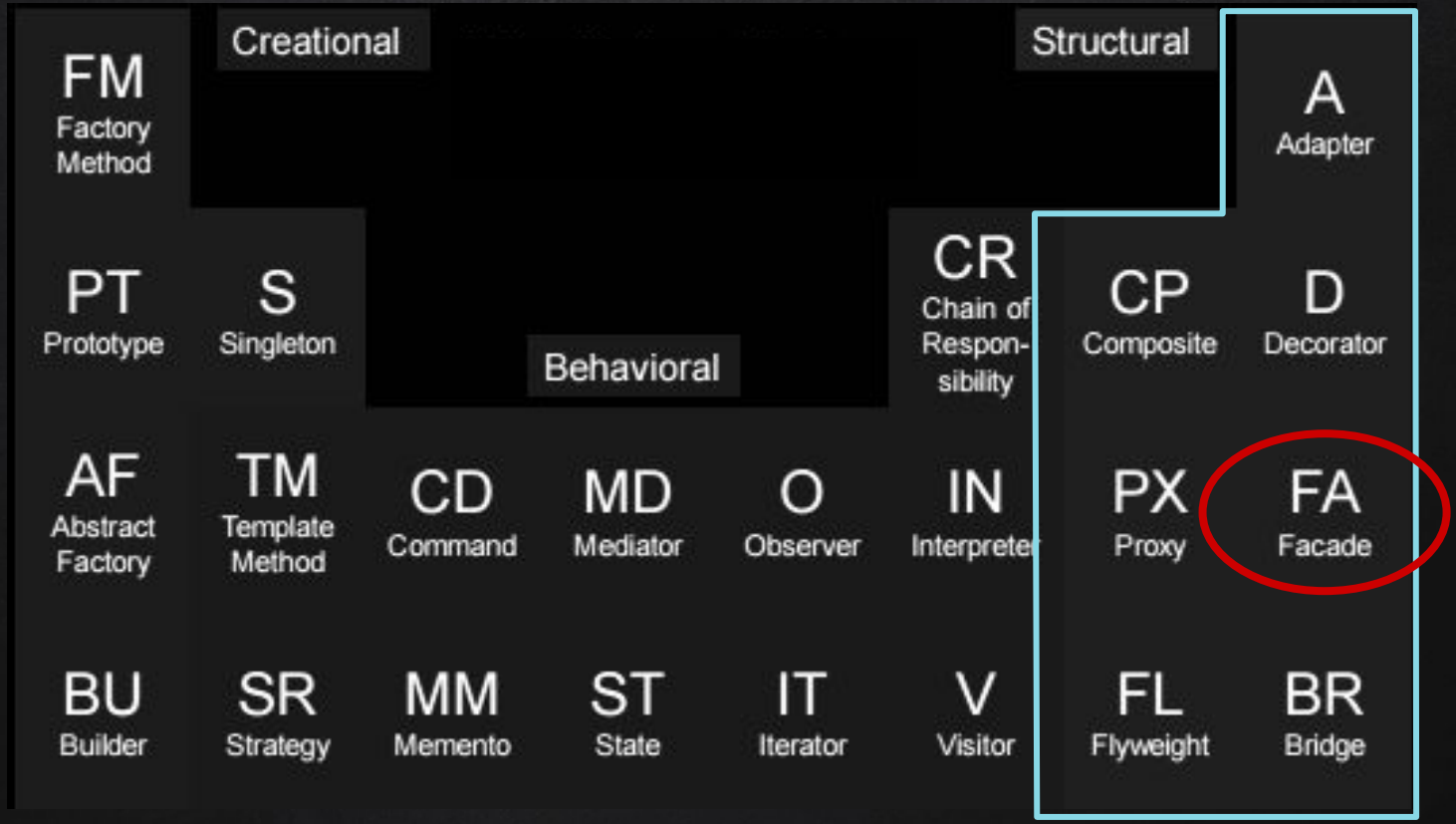
Exemplos da API Java:

```
java.lang.Runtime # getRuntime()
java.awt.Desktop # getDesktop()
```

Resumo: Quando usar singleton?

- Deve haver apenas uma instância de uma classe
- Deve ser acessível aos clientes a partir de um ponto de acesso bem definido
- Quando a única instância deve ser extensível através de subclasses, e os clientes devem poder usar uma instância estendida sem modificar o seu código

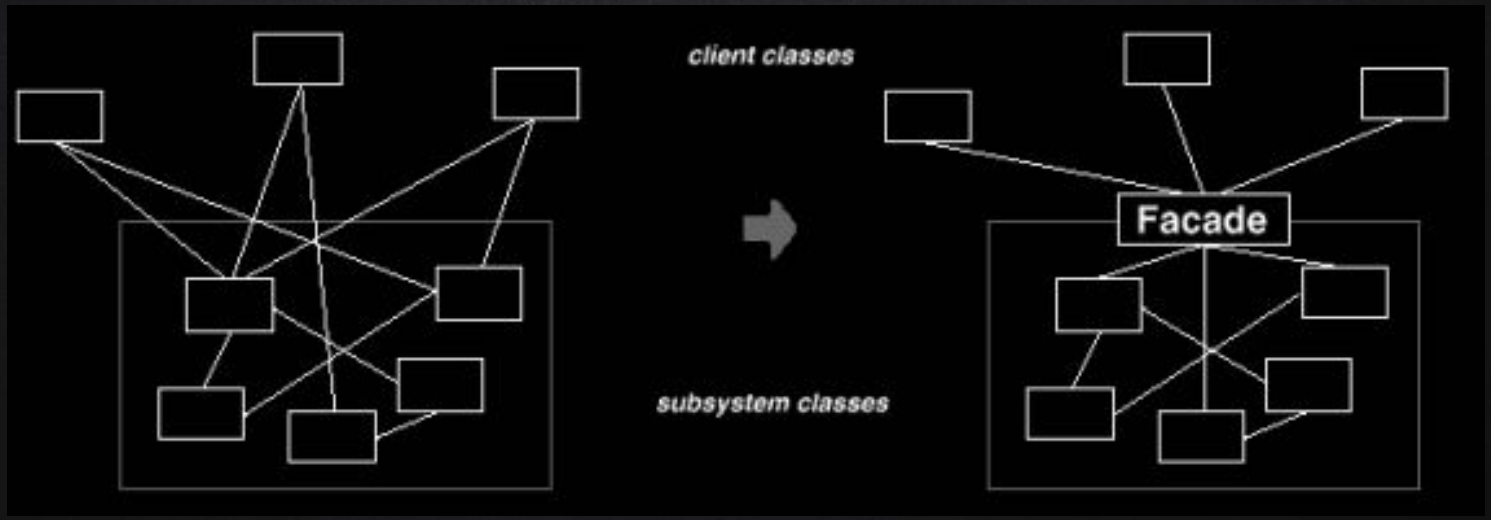
Tipos de padrões



Padrão Facade (structural)



Uma *facade* é uma classe *wrapper* que encapsula um subsistema para ocultar a sua complexidade e actua como um ponto de entrada no subsistema sem adicionar funcionalidade.



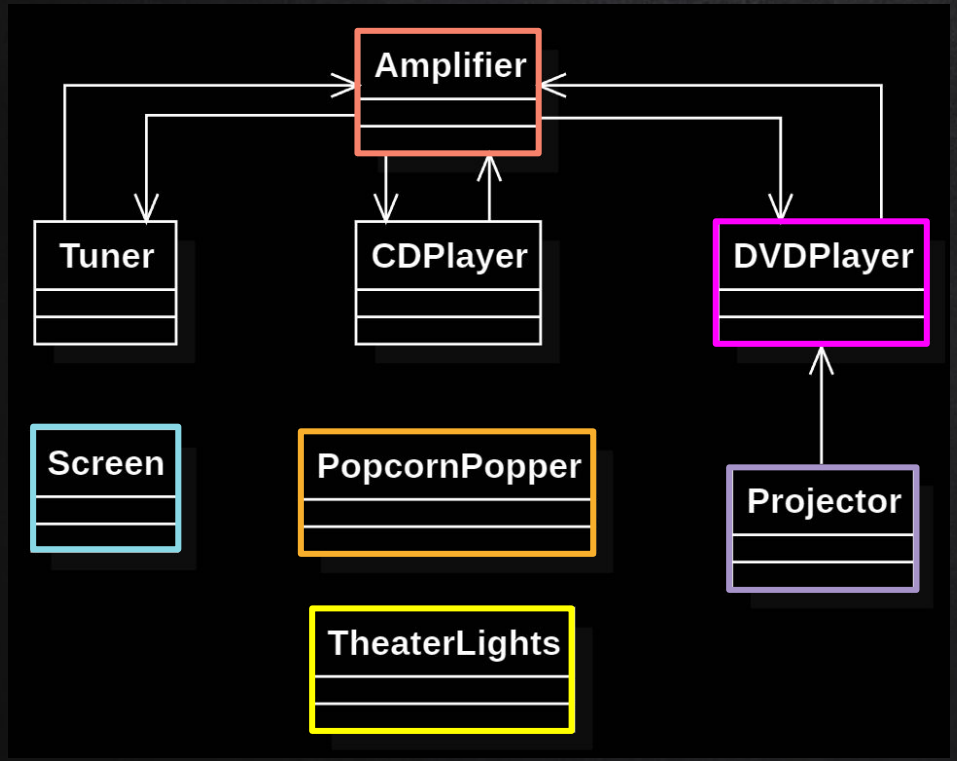
Facade oferece uma interface unificada para um conjunto de interfaces num subsistema

- O objectivo é ocultar a complexidade atrás de uma interface
- *Facade* fornece uma interface unificada para um subsistema complexo (geralmente com um conjunto de interfaces) ou para um grupo de subsistemas
- *Facade* define uma interface de nível superior que torna o(s) subsistema(s) mais fácil(s) de usar
- Estruturar um sistema em subsistemas ajuda a reduzir a complexidade
- Elimina a necessidade de as classes clientes gerirem um subsistema por conta própria, resultando em menos acoplamento entre o subsistema e as classes clientes
- Um dos principais objectivos do desenho é minimizar a comunicação e as dependências entre subsistemas

Etapas de construção do Facade

1. Desenhar a interface *facade*
2. Implementar as classes do subsistema
3. Criar a classe de *facade* e envolver as classes que implementam a interface
4. Utilizar a classe *facade* para aceder ao subsistema

Sistema home cinema



1. Ligar a máquina de pipocas
2. Começar a fazer pipocas
3. Diminuir as luzes
4. Baixar a tela de projecção
5. Ligar o projector
6. Definir o *input* do projector para DVD
7. Colocar o projector no modo de ecrã inteiro
8. Ligar o amplificador de som
9. Definir o amplificador para entrada de DVD
10. Definir o amplificador para som *surround*
11. Definir o volume do amplificador – médio (5)
12. Ligar o leitor de DVD
13. Iniciar a reprodução do leitor de DVD

Sistema home cinema

```
popper.on();
popper.pop();

lights.dim(10);

screen.down();

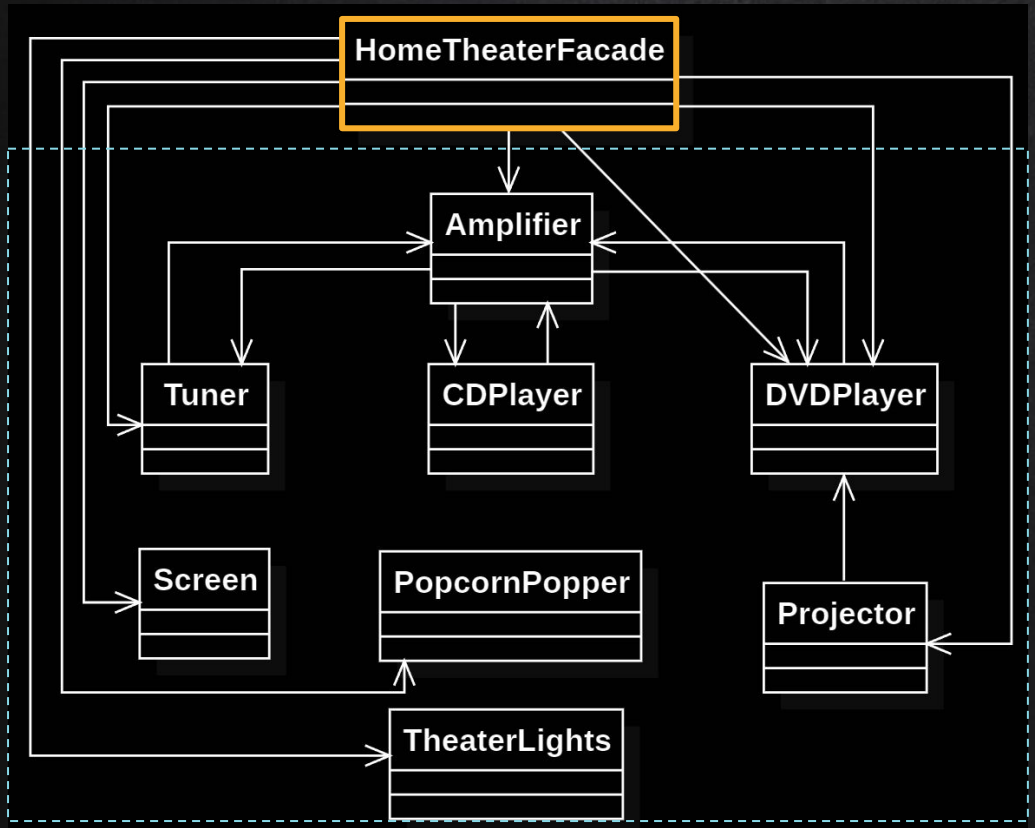
projector.on();
projector.setInput(dvd);
projector.wideScreenMode();

amp.on();
amp.setDvd(dvd);
amp.setSurroundSound();
amp.setVolume(5);

dvd.on();
dvd.play(movie);
```

1. Ligar a máquina de pipocas
2. Começar a fazer pipocas
3. Diminuir as luzes
4. Baixar a tela de projecção
5. Ligar o projector
6. Definir o *input* do projector para DVD
7. Colocar o projector no modo de ecrã inteiro
8. Ligar o amplificador de som
9. Definir o amplificador para entrada de DVD
10. Definir o amplificador para som surround
11. Definir o volume do amplificador – médio (5)
12. Ligar o leitor de DVD
13. Iniciar a reprodução do leitor de DVD

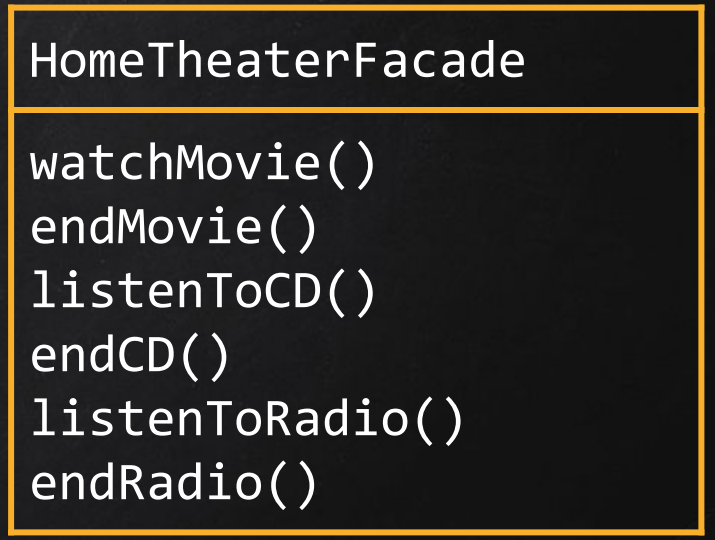
Passo #1 – Desenhar a interface do facade



HomeTheaterFacade
watchMovie() endMovie() listenToCD() endCD() listenToRadio() endRadio()

Passo #1 – Desenhar a interface facade

```
public interface HomeTheaterFacade {
    void watchMovie(String movie);
    void endMovie();
    void listenToCD();
    void endCD();
    void listenToRadio();
    void endRadio();
}
```



Passo #2: Implementar as classes do subsistema

```
public class Amplifier {...}
public class CdPlayer {...}
public class DvdPlayer {...}
public class PopcornPopper {...}
public class Projector {...}
public class Screen {...}
public class TheaterLights {...}
public class TheaterLights {...}
```

Passo #3: Criar a classe *facade* e envolver as classes que implementam a interface

```
public class HomeTheaterFacadeClass implements HomeTheaterFacade {
   private Amplifier amp;
   Tuner tuner;
   DvdPlayer dvd;
   CdPlayer cd;
   Projector projector;
   TheaterLights lights;
   Screen screen;
   PopcornPopper popper;
   public HomeTheaterFacadeClass(Amplifier amp, Tuner tuner, DvdPlayer dvd,
                                 CdPlayer cd, Projector projector,
                                 Screen screen, TheaterLights lights,
                                 PopcornPopper popper) {

      ...
   }
}
```

Passo #3: Criar a classe *facade* e envolver as classes que implementam a interface

```
public class HomeTheaterFacadeClass implements HomeTheaterFacade {
    ...
    public void watchMovie(String movie) {
        System.out.println("Get ready to watch a movie...");
        popper.on();
        popper.pop();
        lights.dim(10);
        screen.down();
        projector.on();
        projector.wideScreenMode();
        amp.on();
        amp.setDvd(dvd);
        amp.setSurroundSound();
        amp.setVolume(5);
        dvd.on();
        dvd.play(movie);
    }
}
```

Passo #3: Criar a classe *facade* e envolver as classes que implementam a interface

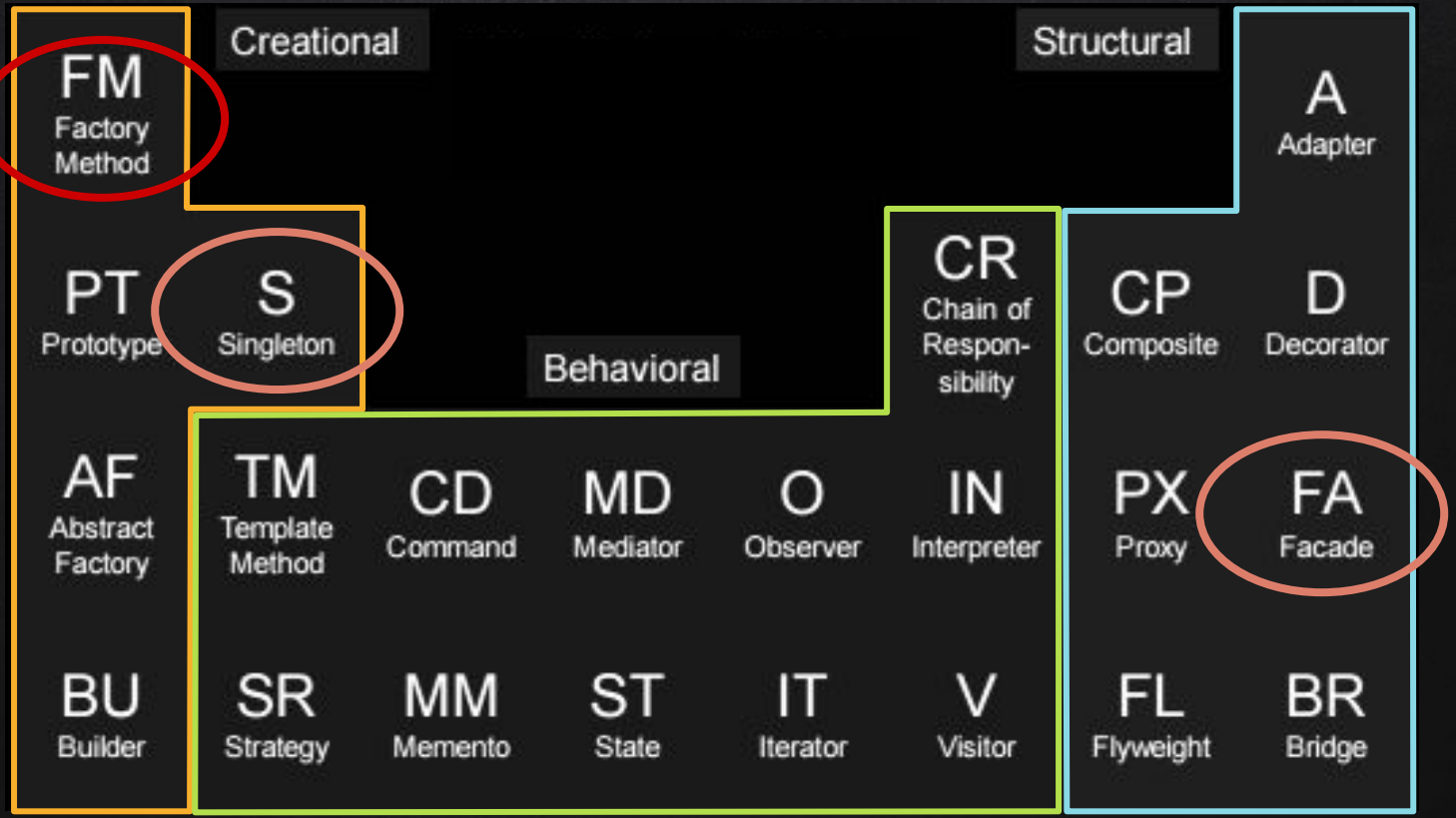
```
public class HomeTheaterFacadeClass implements HomeTheaterFacade {
    ...
    public void endMovie() {
        System.out.println("Shutting movie theater down...");
        popper.off();
        lights.on();
        screen.up();
        projector.off();
        amp.off();
        dvd.stop();
        dvd.eject();
        dvd.off();
    }

}
```

Passo #4: Usar a classe *Facade* para aceder ao subsistema

```
public class HomeTheaterTestDrive {
    public static void main(String[] args) {
        // instantiate components here
        Amplifier amp = new Amplifier();
        Tuner tuner = new Tuner();
        DvdPlayer dvd = new DvdPlayer();
        CdPlayer cd = new CdPlayer();
        Projector projector = new Projector();
        TheaterLights lights = new TheaterLights();
        Screen screen = new Screen();
        PopcornPopper popper = new PopcornPopper();
        HomeTheaterFacadeClass homeTheater =
                new HomeTheaterFacadeClass(amp, tuner, dvd, cd,
                        projector, screen, lights, popper);
        homeTheater.watchMovie("Raiders of the Lost Ark");
        homeTheater.endMovie();
    }
}
```

Tipos de padrões



Padrão factory method (creational)

Tem como objectivo ocultar a criação de instâncias de um determinado tipo (produtos) por trás de uma interface (criador)

Desta forma, o código do cliente não depende de classes concretas

Podemos substituir uma classe concreta de *produto* por outra sem impacto no código do cliente



Primeiro: o que é um Factory Object? (não GoF)



Loja de Facas



Faca para Carne



Faca de *Chef*

Primeiro: o que é um Factory Object? (não GoF)



Loja de Facas

`orderKnife(knifeType)`





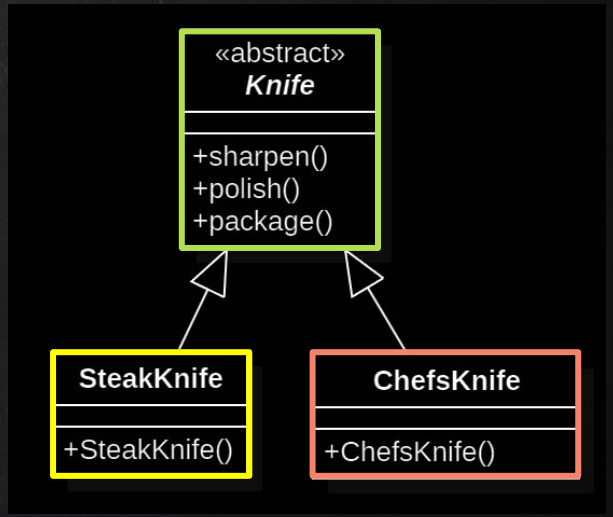
Faca para Carne



Faca de *Chef*

Primeira tentativa

```
Knife orderKnife(String knifeType) {
    Knife knife;
    //create knife object concrete instantiation
    if (knifeType.equals("steak")) {
        knife = new SteakKnife();
    } else if (knifeType.equals("chefs")) {
        knife = new ChefsKnife();
    }
    knife.sharpen();
    knife.polish();
    knife.package();
    return knife;
}
```



Primeiro: o que é um Factory Object? (não GoF)



Loja de Facas

`orderKnife(knifeType)`

E se adicionarmos mais facas?





Faca para Carne





Faca de *Chef*

Primeiro: o que é um Factory Object? (não GoF)



Loja de Facas

```
orderKnife(knifeType)
```

E se houver clientes que precisem de criar facas?





Faca para Carne



Faca de *Chef*



Primeiro: o que é um Factory Object?



Loja de Facas

```
orderKnife(knifeTyp
```

E se houver clientes qu
precisem de criar facas



Vamos criar facas noutro sítio!



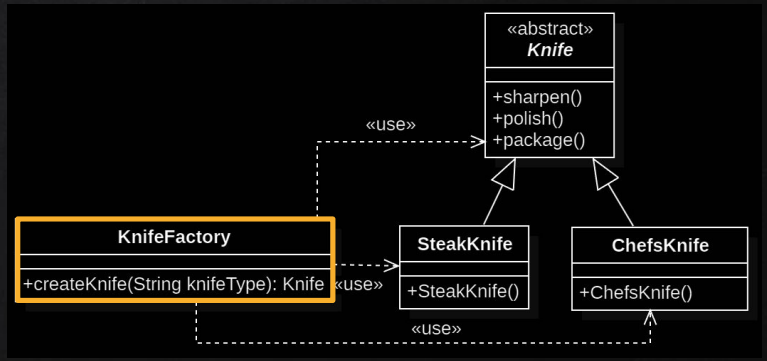
Faca de *Chef*

Factory Object

cria objectos de produto de tipos específicos

```
public class KnifeFactory {
   public Knife createKnife(String knifeType) {
      Knife knife = null;
      // create Knife object
      if (knifeType.equals(“steak”)) {
         knife = new SteakKnife();
      } else if (knifeType.equals(“chefs”)) {
         knife = new ChefsKnife();
       }
       return knife;
   }
}
```

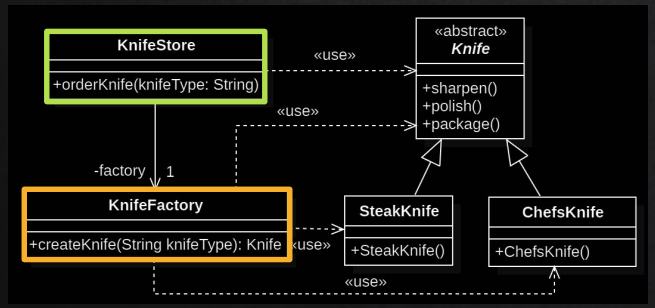




Factory Object usado por KnifeStore

```
public class KnifeStore {
   private KnifeFactory factory;
   // require a KnifeFactory object to be passed
   // to this constructor:
   public KnifeStore(KnifeFactory factory) {
      this.factory = factory;
   }
   public Knife orderKnife(String knifeType) {
      …
   }
}
```

Nova abordagem




```
public Knife orderKnife(String knifeType) {
   Knife knife;
   //use the create method in the factory
   knife = factory.createKnife(knifeType);
   // prepare the Knife
   knife.sharpen();
   knife.polish();
   knife.package();
   return knife;
}
```

Benefícios

Ao usar um *factory object*, se existirem vários clientes que pretendam instancionar o mesmo conjunto de classes, elimina-se código reduntante e torna-se o software mais fácil de modificar.

O padrão Factory Object não é GoF!

Vamos ver o que GoF propõe...

Padrão Factory Method (creational)



Padrão Factory Method (creational)

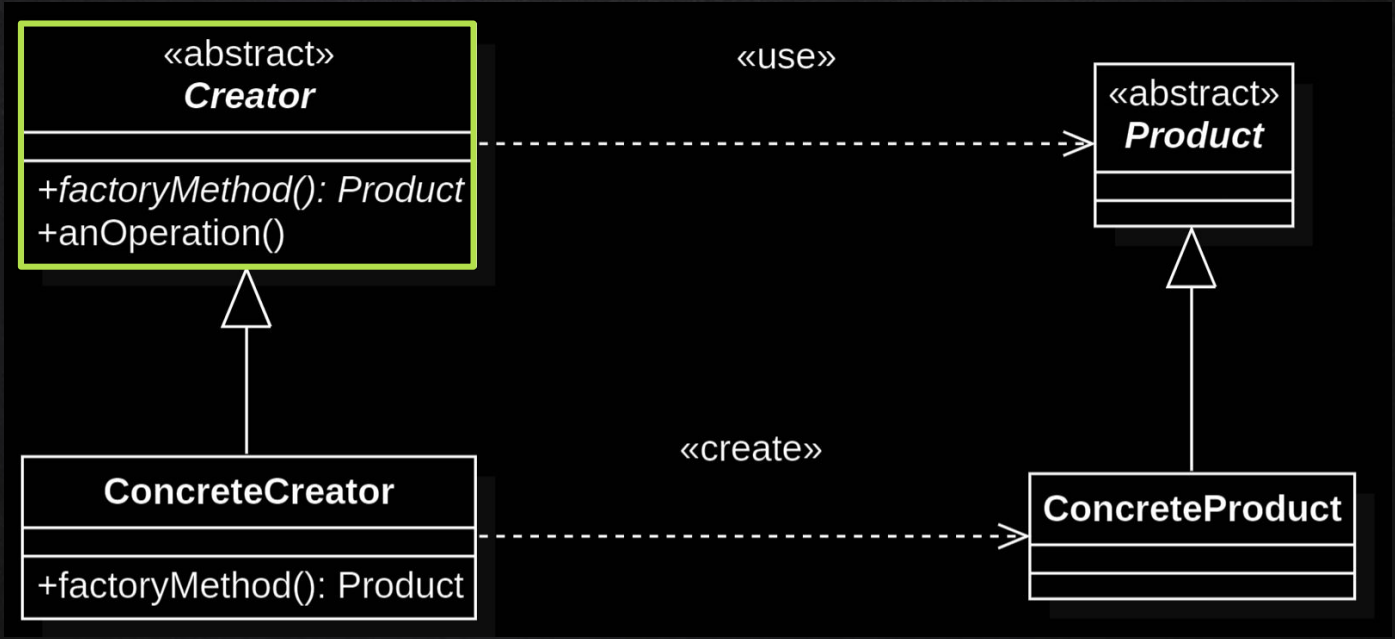
O objetivo é ocultar a criação de instâncias de um determinado tipo (produtos) por trás de uma interface (criador)

Desta forma, o código do cliente não depende de classes concretas

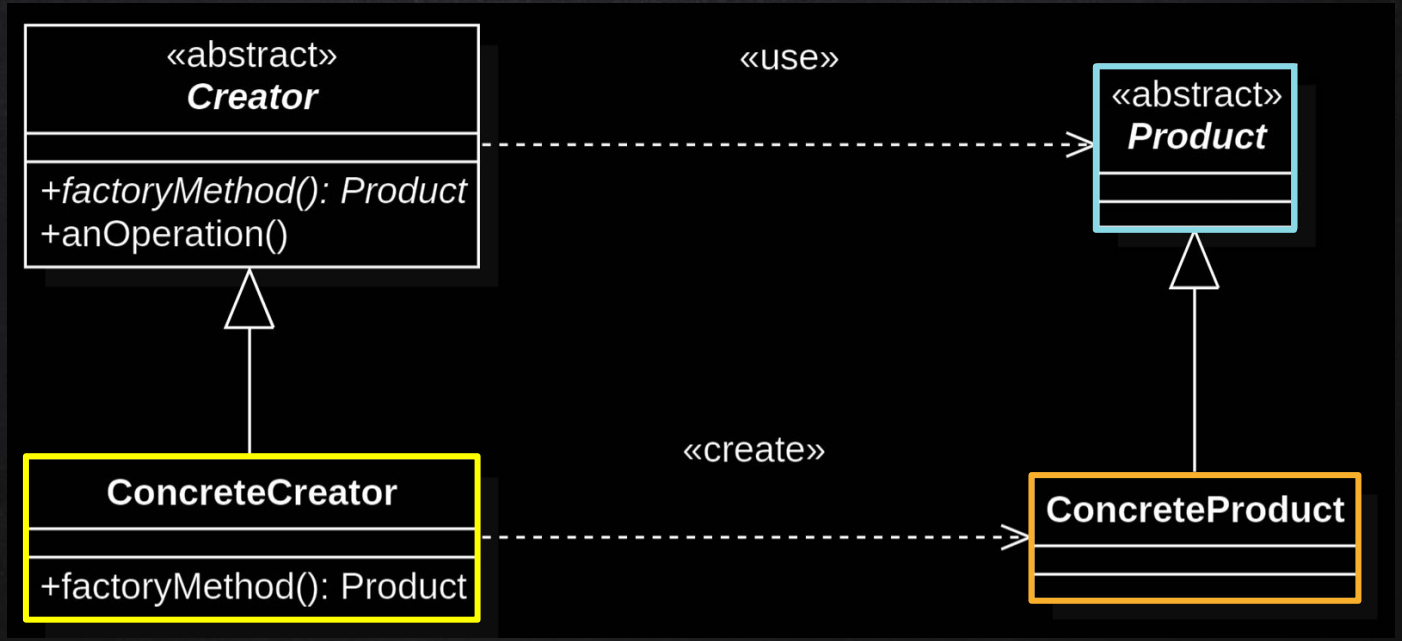
Podemos substituir uma classe concreta de produto por outra sem impacto no código do cliente



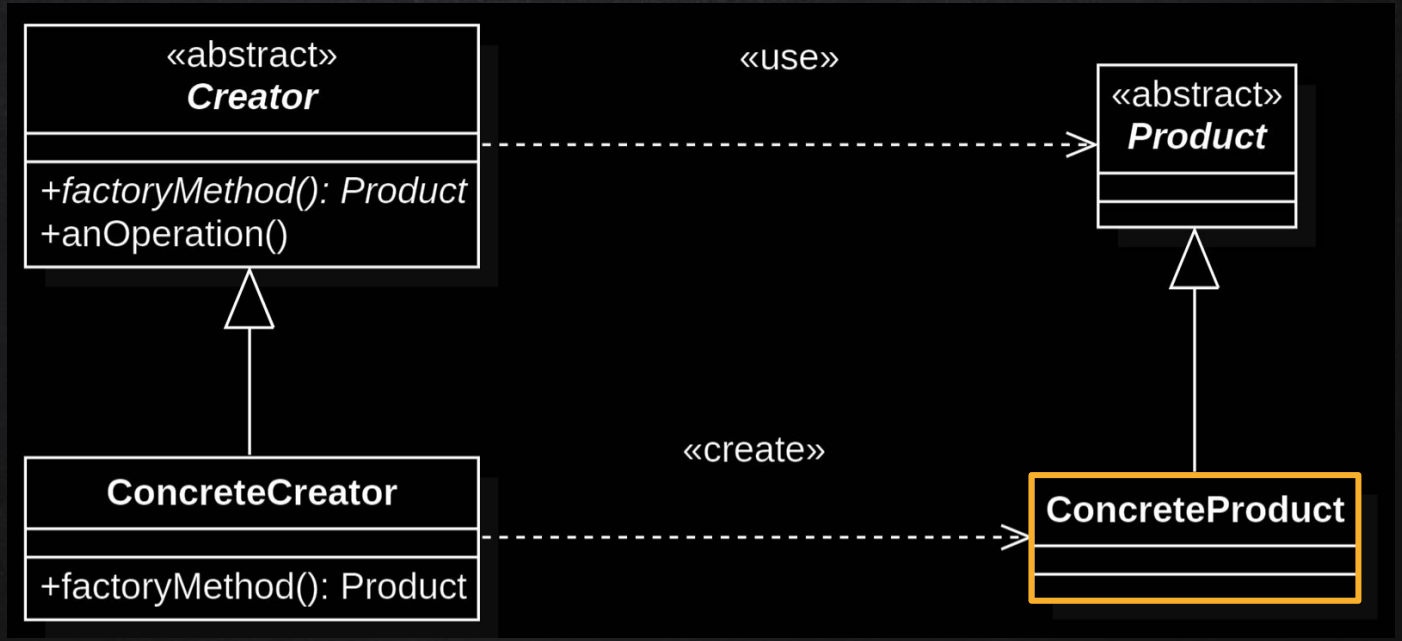
Creator declara um método que devolve uma instância de várias classes possíveis da hierarquia, dependendo dos dados recebidos



ConcreteCreator é uma classe que tem o código real que instancia objectos *Product*. Apenas este código depende das classes ConcreteProduct.

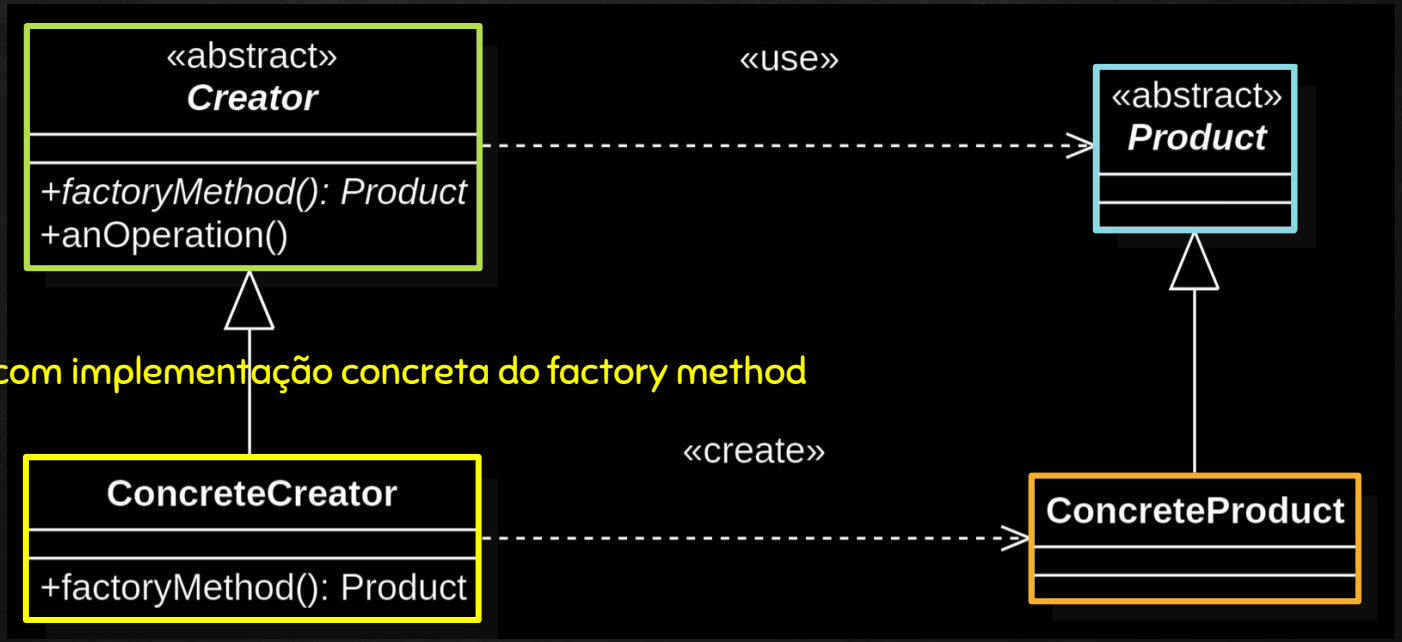


Cada classe ConcreteProduct executa uma tarefa de forma diferente e/ou é otimizada para diferentes tipos de dados.



Quando o cliente (a vossa aplicação) cria um produto, a aplicação não o cria directamente. Por outro lado, a aplicação solicita à factory que crie o produto invocando `factoryMethod()`. Dependendo da concrete factory que for utilizada, é criado um `ConcreteProduct`.

Interface com o factory method (permite chamar a *factory*)



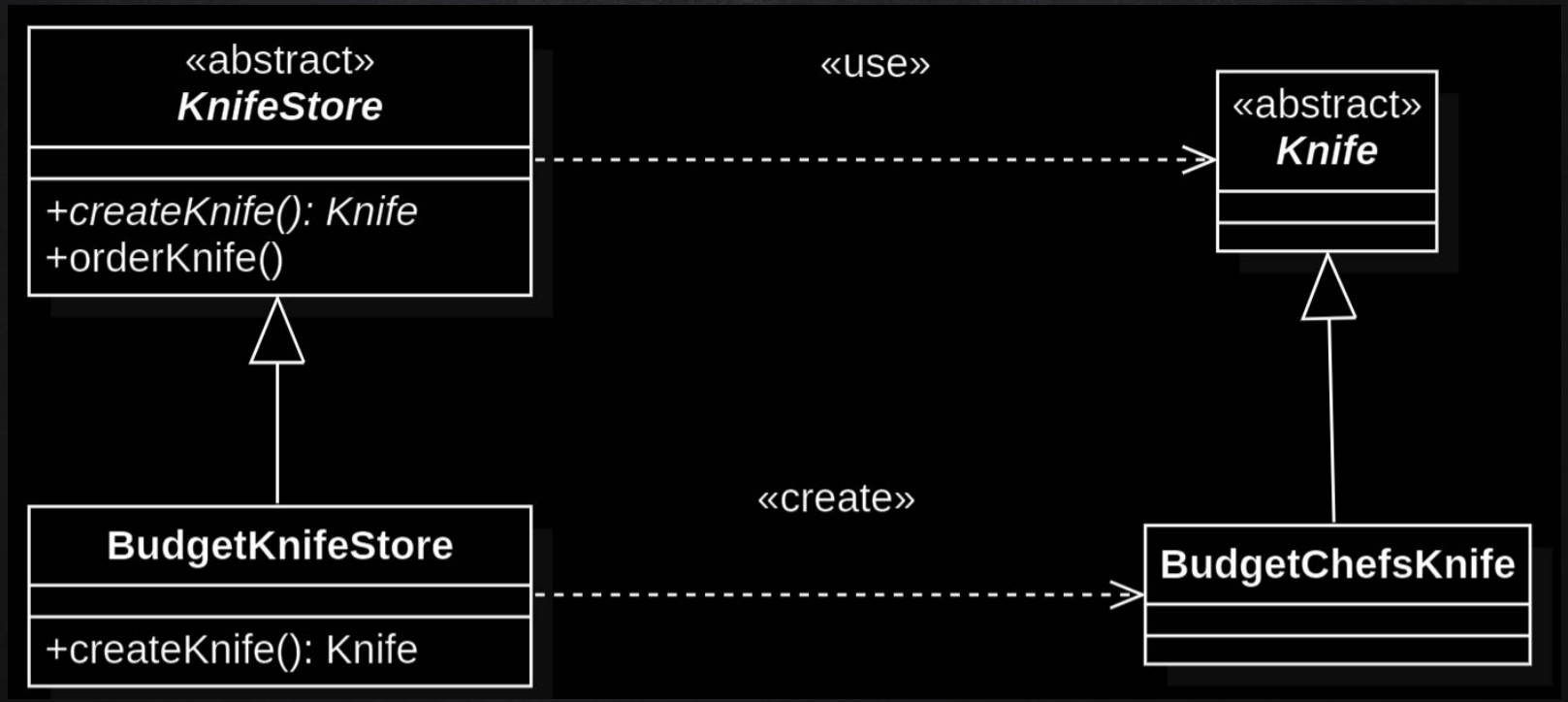
Analogia: pense numa pizzaria

A pizzaria tem um menu (a interface) e diferentes *chefs* (as fábricas concretas), onde cada um é especialista em fazer um tipo específico de pizza (o produto).

Quando um cliente pede uma pizza, ele não vai até a cozinha e faz a pizza: pede ao *chef* para a preparar.

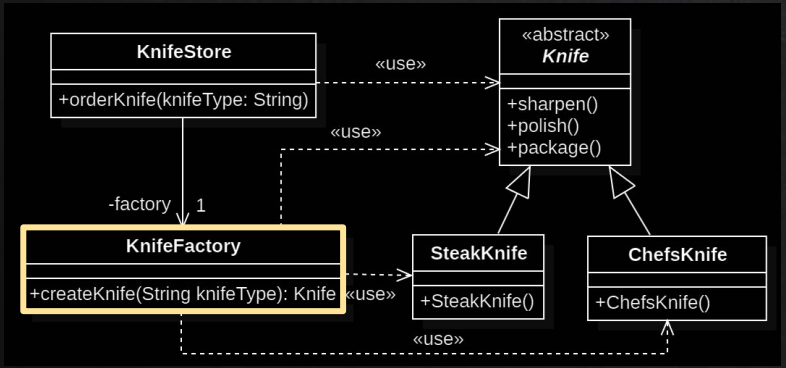
Dependendo do *chef*, receberá um tipo diferente de pizza.

O exemplo da loja de facas com o padrão Factory Method

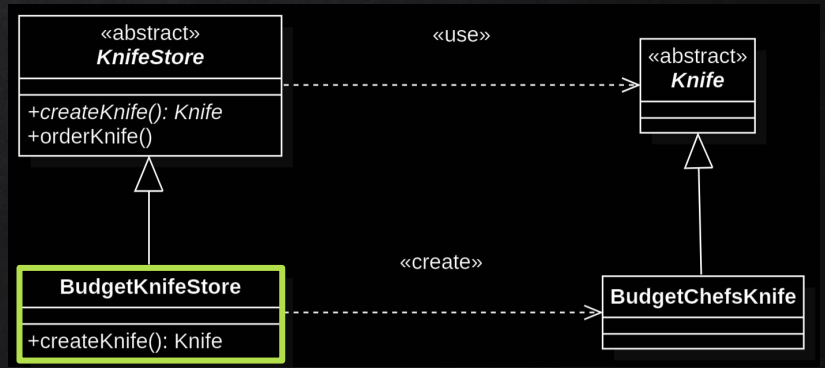


Tanto o padrão Factory Method quanto o padrão Factory Object são usados para criar objetos. A escolha dependerá dos vossos requisitos específicos e objetivos de desenho.

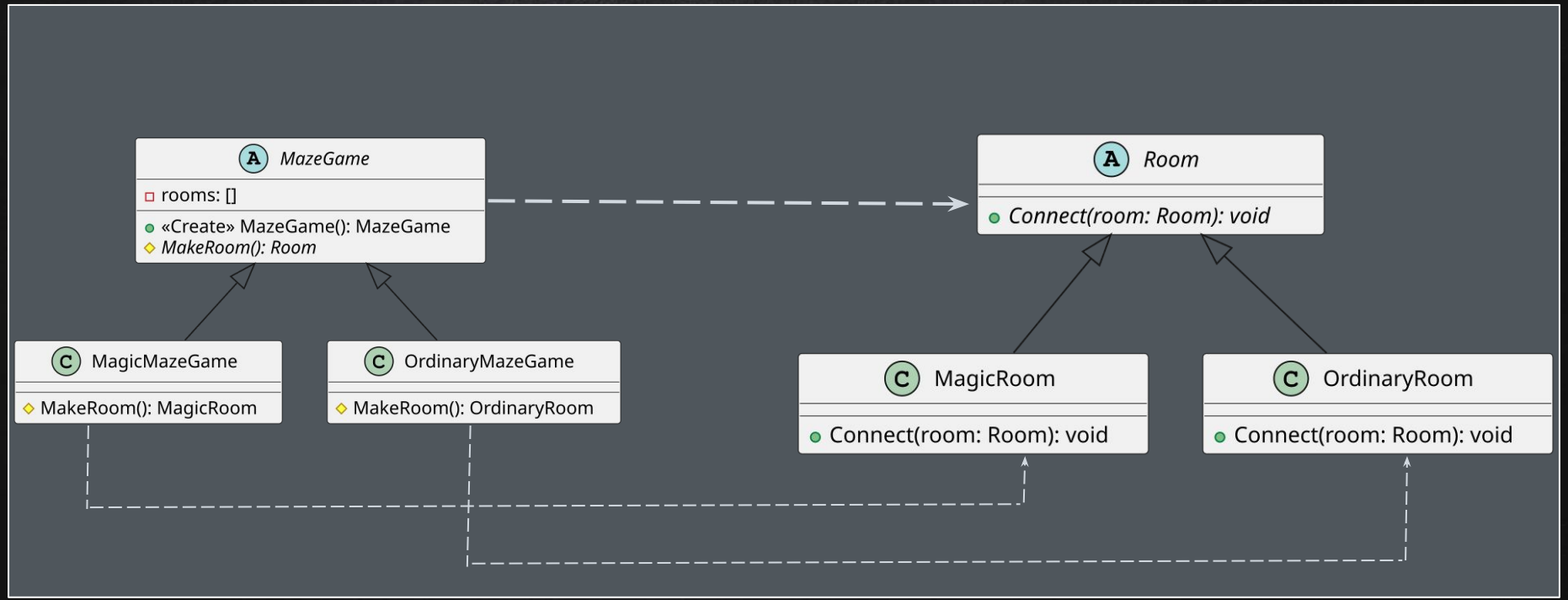
O padrão *Factory Object* separa a responsabilidade de criar objetos numa classe ou *factory object* dedicado



O padrão *Factory Method* depende de subclasses para criar objetos.



Outro exemplo – Magic Rooms



https://en.wikipedia.org/wiki/Factory_method_pattern